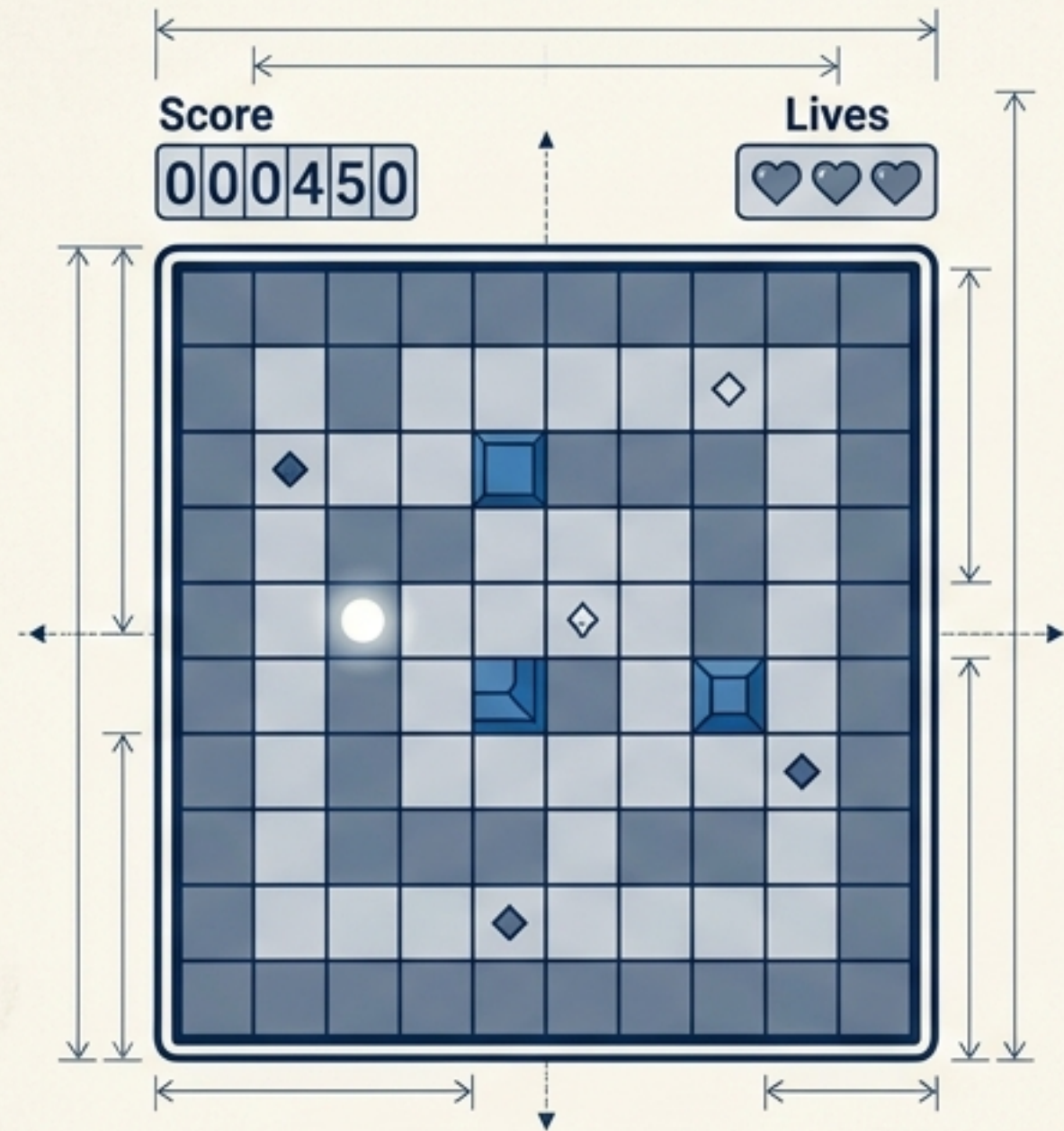


El Laberinto de los Tesoros: Anatomía de un Juego en C#

```
#####  
#   . . . . #   . . . . #   . . . . . . . . * #  
#   . #   . # *   . . . #   #####. #  
#   . #   #####. #   . . . . #. #  
#   . #   . . . . . #   . ## * #. #. #  
#   #####. #####. # * #. #  
#   . . . . #. #   . #. #. #  
#   . #   . #. #   @   #####. #. #  
#   . #   . # * #   #   . . . * #. #  
#   . #   . # * #####. . . * #. #  
#   . #   . #####. . . . . #####. #  
#   . #   . * * . . . . *   . . * . . . #. #  
#   #####. #####. #. #  
#   . . . . . . . . . . *   . . . . * #  
#####
```

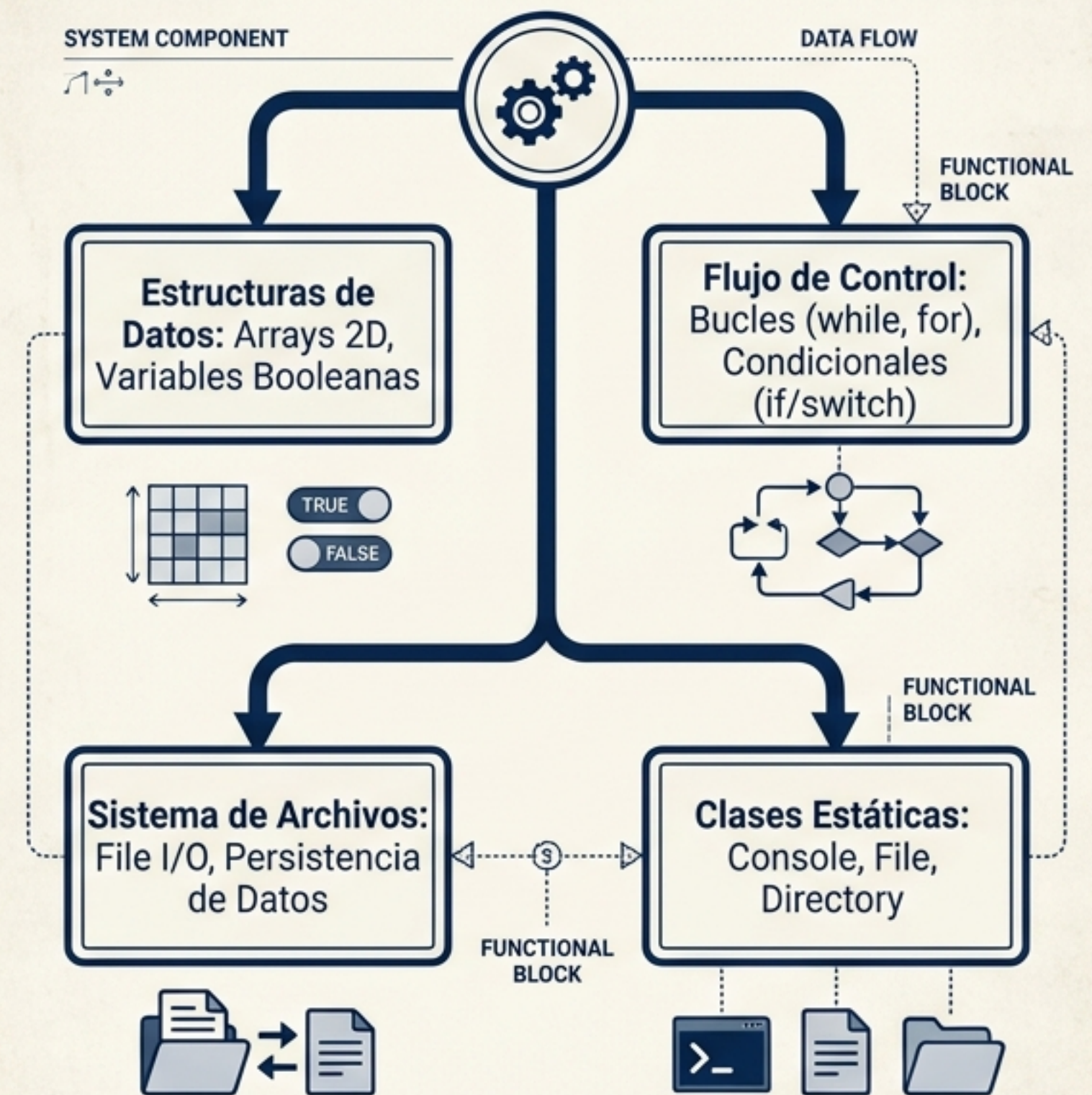
De la lógica base a la pantalla. Guía visual de implementación para estudiantes de programación.

¿Qué estamos construyendo?

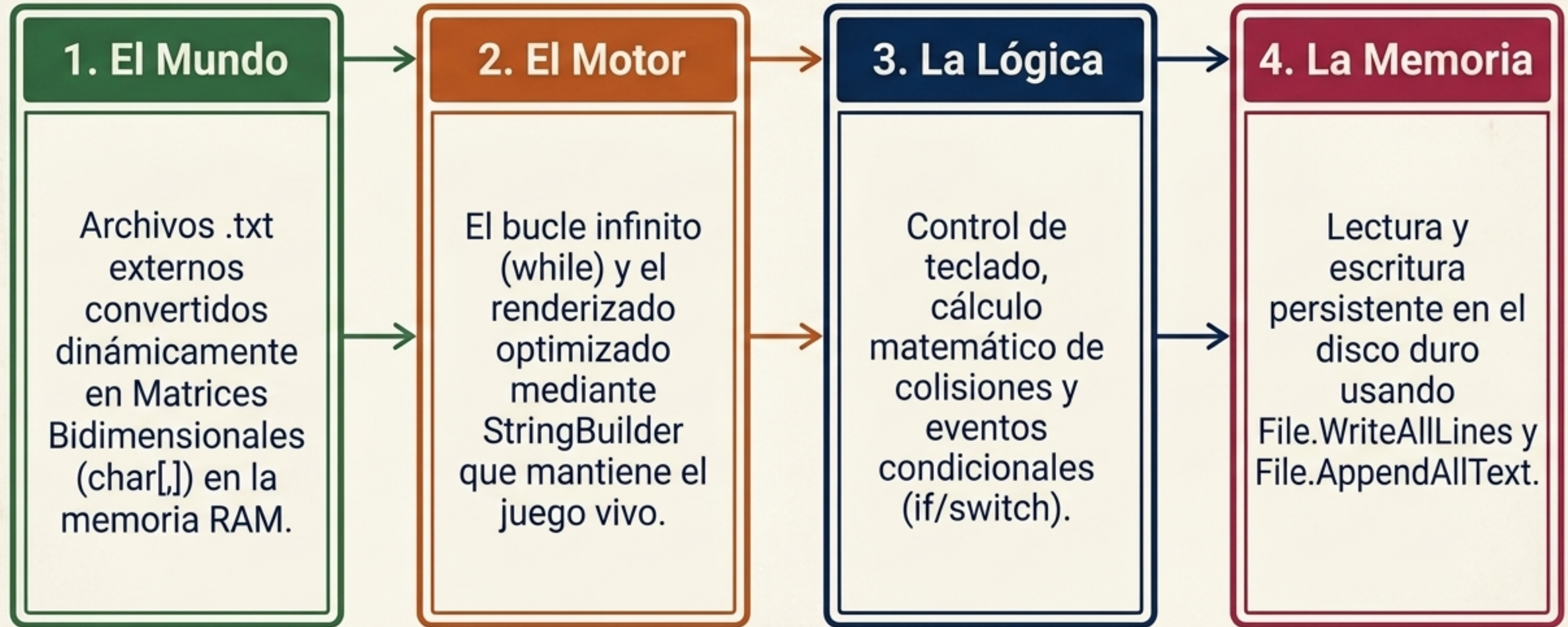


Un juego de laberintos persistente, sin parpadeos visuales, con guardado de puntuaciones y mapas editables.

The Architectural Mapping Diagram



Arquitectura del Sistema: Los Cuatro Motores



Construyendo el Mundo: Leyenda de Constantes

CÓDIGO (CONSTANTES C++)	SÍMBOLO	SIGNIFICADO
<code>const char PARED = '#';</code>	#	Muro infranqueable
<code>const char VACIO = ' ';</code>	[Espacio]	Suelo transitable en RAM
<code>const char SUELO_FICHERO = '.';</code>	.	Suelo transitable en disco
<code>const char JUGADOR = '@';</code>	@	El avatar del usuario
<code>const char TESORO = '*';</code>	*	Incre

El Poder de las Constantes

¿Por qué no escribir '#' directamente en el código? Usar constantes hace el código legible y permite rediseñar todo el aspecto visual cambiando un solo carácter en la cabecera.

De Archivo a Matriz Bidimensional

```
#####  
#.#.....#.#  
#.#.####.####  
#.....#  
#####
```

nivel1.txt

`File.ReadAllLines()`

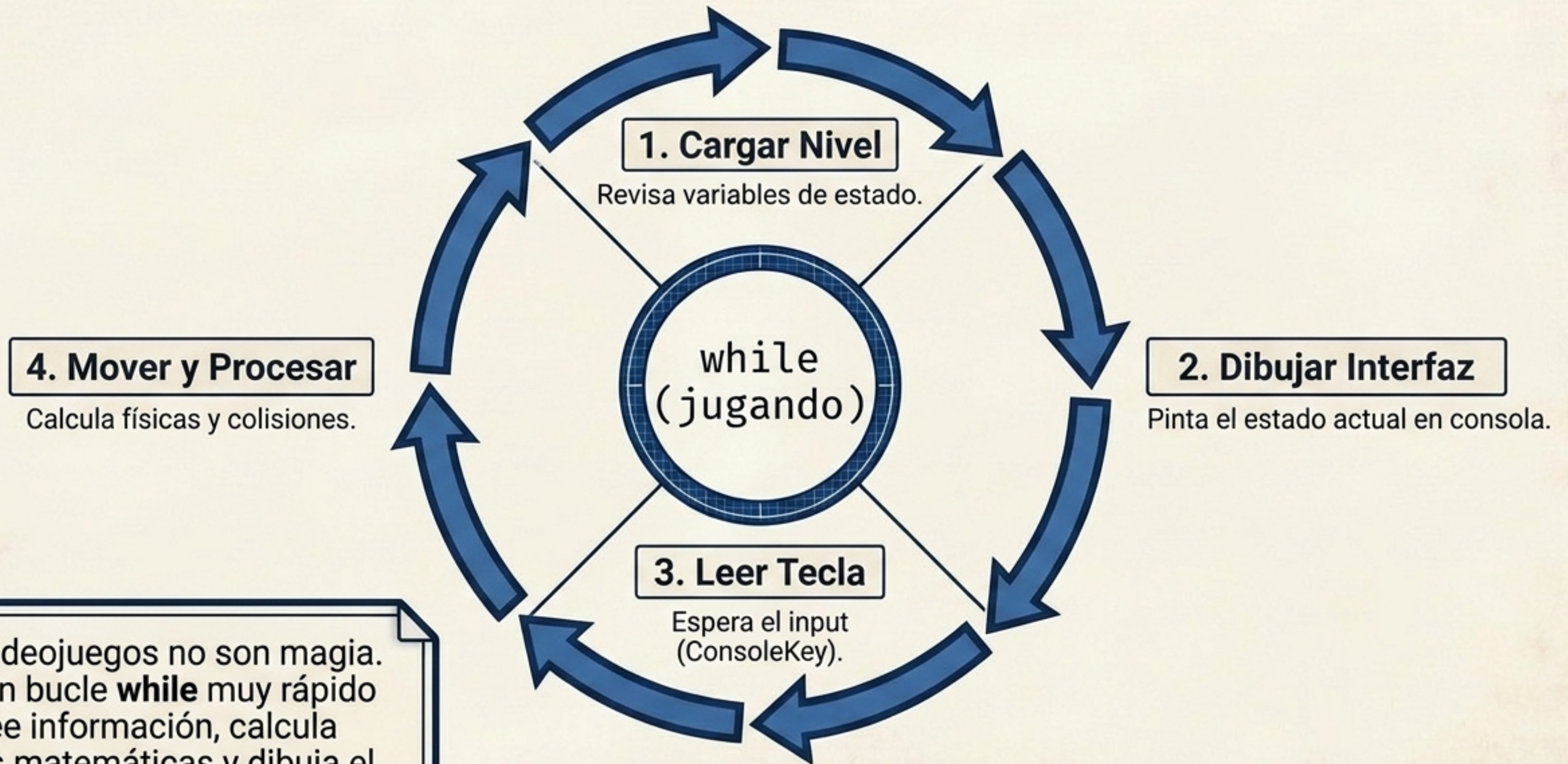


#	#	#	#	#	#
#					#
#					#
#	#	#	#	#	#
#					#
#	#	#	#	#	#

`char[,]` mapa

El juego carga mapas diseñados en texto plano. En el disco, usamos el punto '.' para el suelo (evitando errores por espacios invisibles en el bloc de notas). Al cargarlo en la memoria RAM, el método sustituye los puntos por espacios vacíos para limpiar la pantalla.

El Game Loop: El Corazón del Sistema



Los videojuegos no son magia. Son un bucle **while** muy rápido que lee información, calcula reglas matemáticas y dibuja el resultado una y otra vez.

Evitando el Parpadeo (Flickering)

El Método Novato

`Console.Clear()`

```
Console.Clear();
```

Problema

Borra completamente la pantalla y la reescribe desde cero. Causa un parpadeo visual constante que lastima la vista porque la consola queda vacía por un milisegundo.

El Método Profesional

`Console.SetCursorPosition(0,0)`

```
Console.SetCursorPosition(0,0);  
Console.Write(pantalla.ToString());
```

Solución

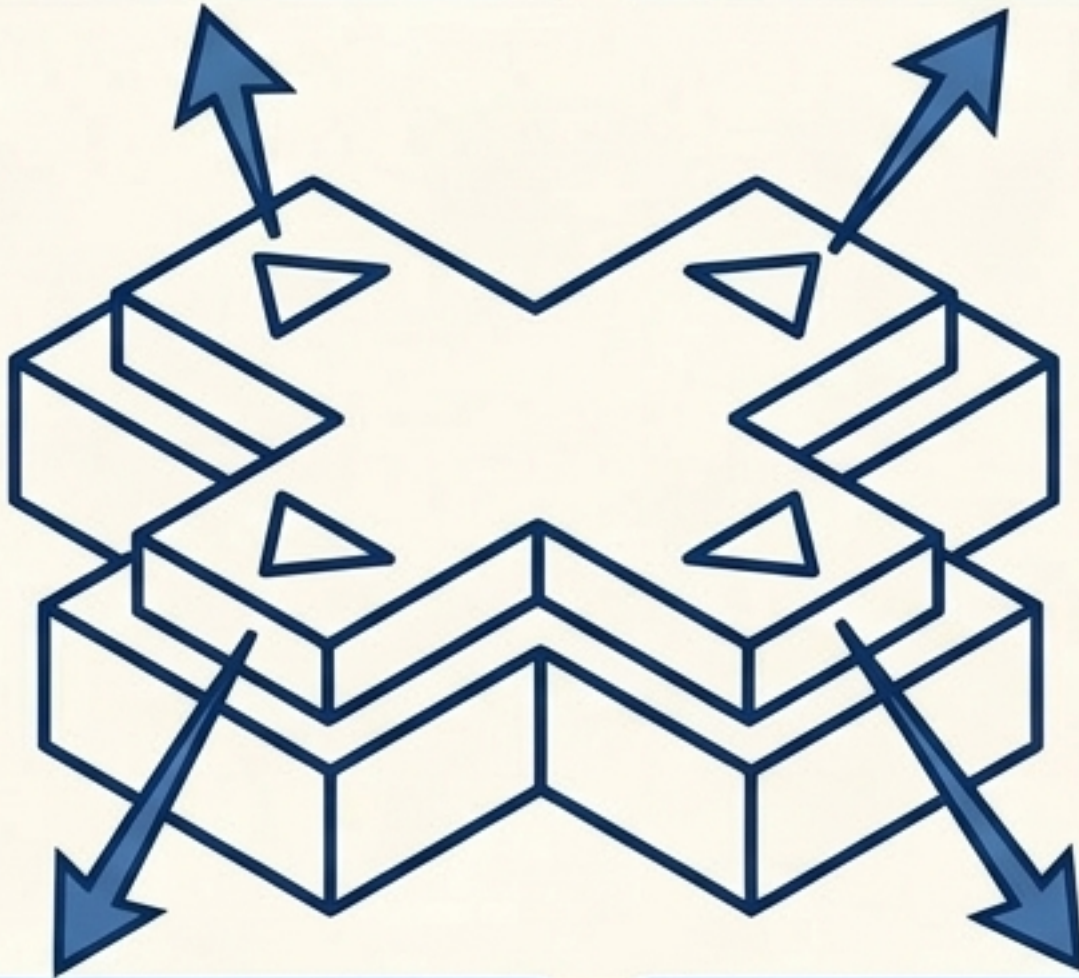
El cursor vuelve al inicio de la consola y sobrescribe el texto existente de un solo golpe leyendo desde un buffer de memoria.

El uso de `StringBuilder` es vital aquí. Permite ensamblar grandes bloques de texto eficientemente en la memoria antes de enviarlos a la consola de una sola vez.

Lógica de Movimiento: Cálculo de Destino

```
case ConsolkeyKey.UpArrow:  
    destinoY--;  
    break;
```

```
case ConsolkeyKey.RightArrow:  
    destinoX++;  
    break;
```



```
case ConsolkeyKey.LeftArrow:  
    destinoX--;  
    break;
```

```
case ConsolkeyKey.DownArrow:  
    destinoY++;  
    break;
```

El modificador **ref**

El método `CalcularDestino` recibe **ref int destinoX**, **ref int destinoY**. No queremos trabajar con copias desechables de las coordenadas; la palabra clave **ref** permite que el método modifique directamente las variables originales en la memoria.

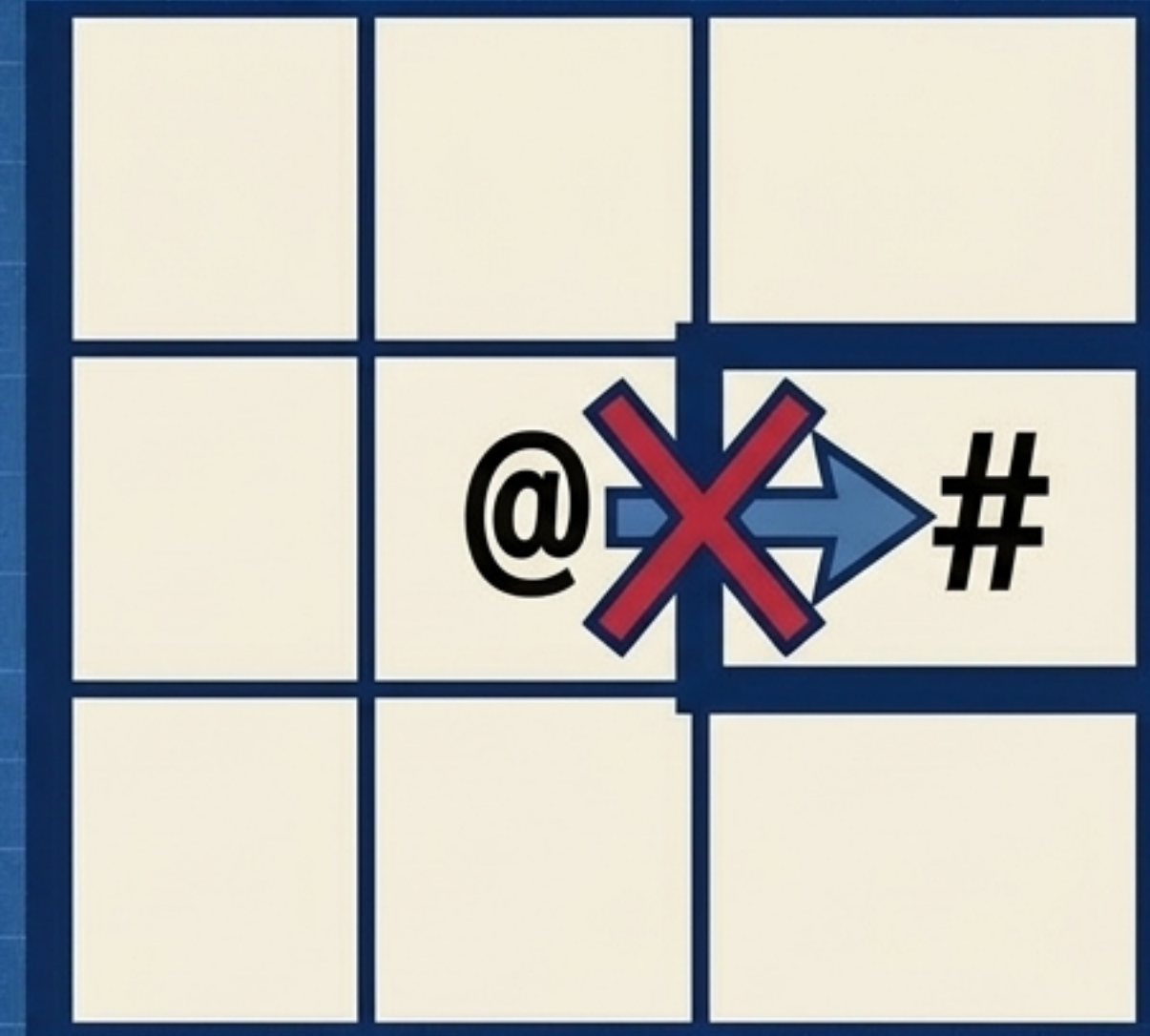
Reglas de Colisión: Validando el Movimiento

IntentarMoverJugador

¿Está dentro de la matriz?
Evita errores de Index Out of Bounds.

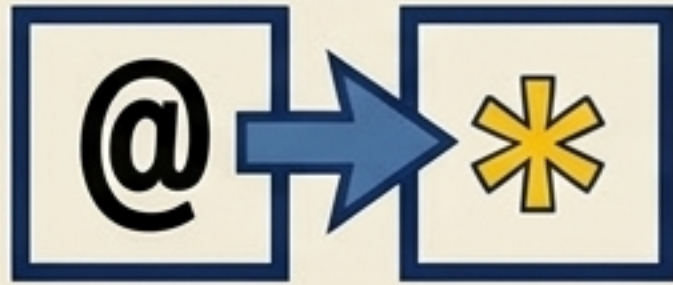
¿Es una pared (#)?
Bloquea el paso físico.

¿Es una puerta (P) sin llave?
Bloquea el acceso lógico.



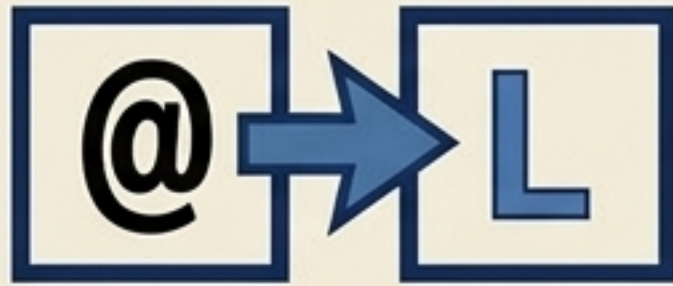
Solo si el movimiento sobrevive a este triple filtro lógico, las coordenadas reales del jugador se actualizan.

Eventos en la Casilla: Mutación del Estado



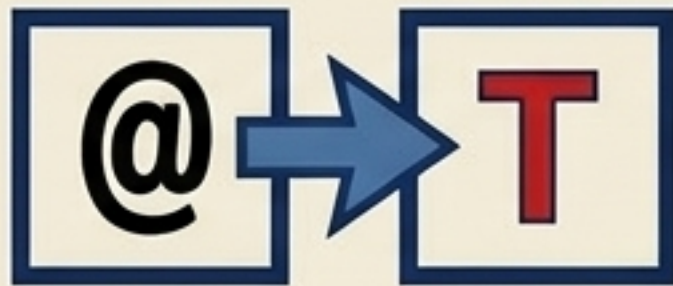
`tesorosRecolectados++;`

Incrementa el contador matemático de la partida.



`tieneLlave = true;`

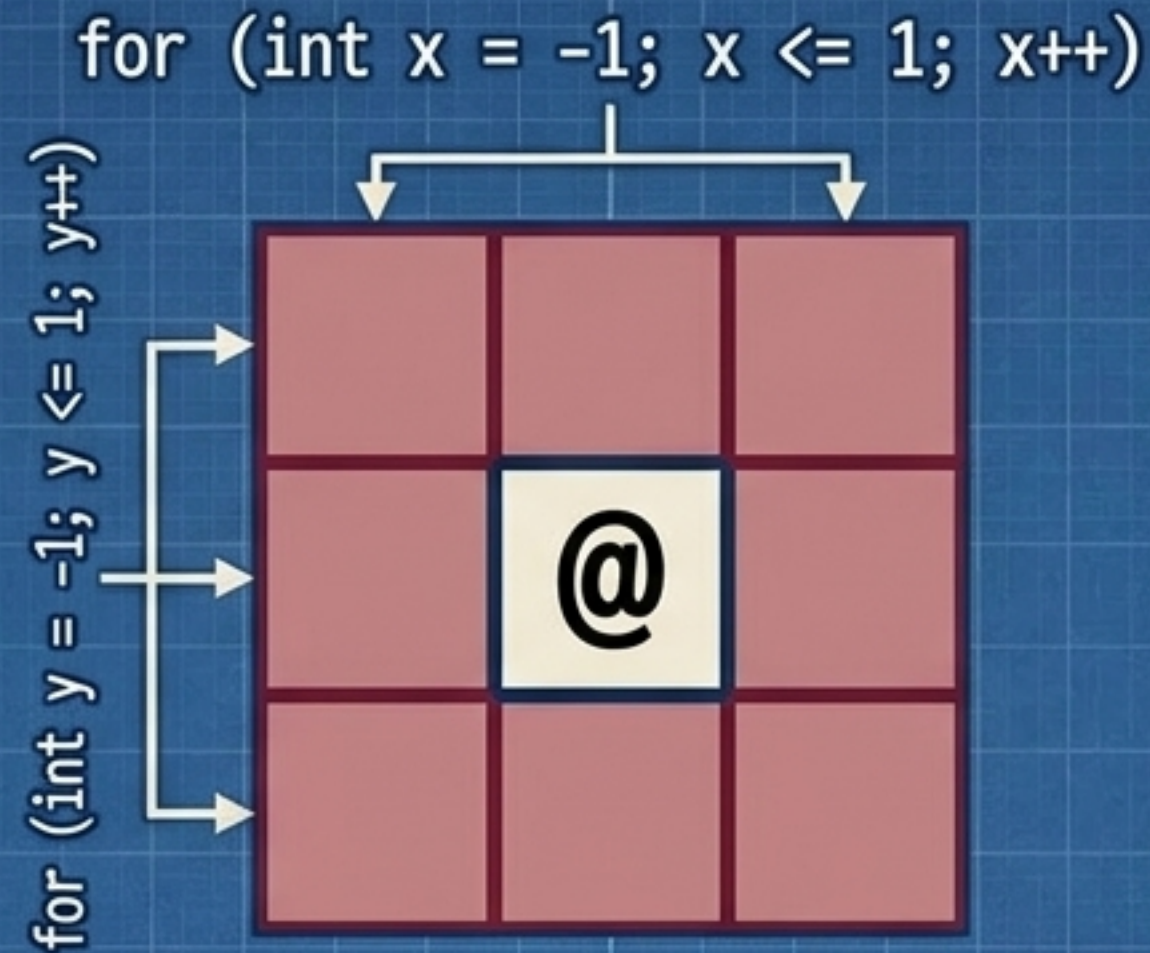
Altera un interruptor booleano global, cambiando las reglas de colisión para permitir abrir puertas.



`vidas--;`
`jugadorX = inicioX;`

Aplica un castigo numérico y sobrescribe las coordenadas del jugador, teletransportándolo al inicio.

La Mecánica de la Bomba: Bucles Anidados



¿Cómo destruimos paredes en un radio de 1 casilla?

Se utilizan dos bucles for anidados. Uno itera la fila (y) desde -1 hasta +1 respecto a la posición central. El otro itera la columna (x) de la misma manera.

Regla de Seguridad: Condicionales internos evitan que el radio destruya los bordes exteriores de la matriz, impidiendo que el jugador escape hacia el vacío.

Resiliencia: Programación Defensiva



Path A (Normal)

Archivos .txt detectados

CargarMapaDesdeFichero()

Path B (Fallback)

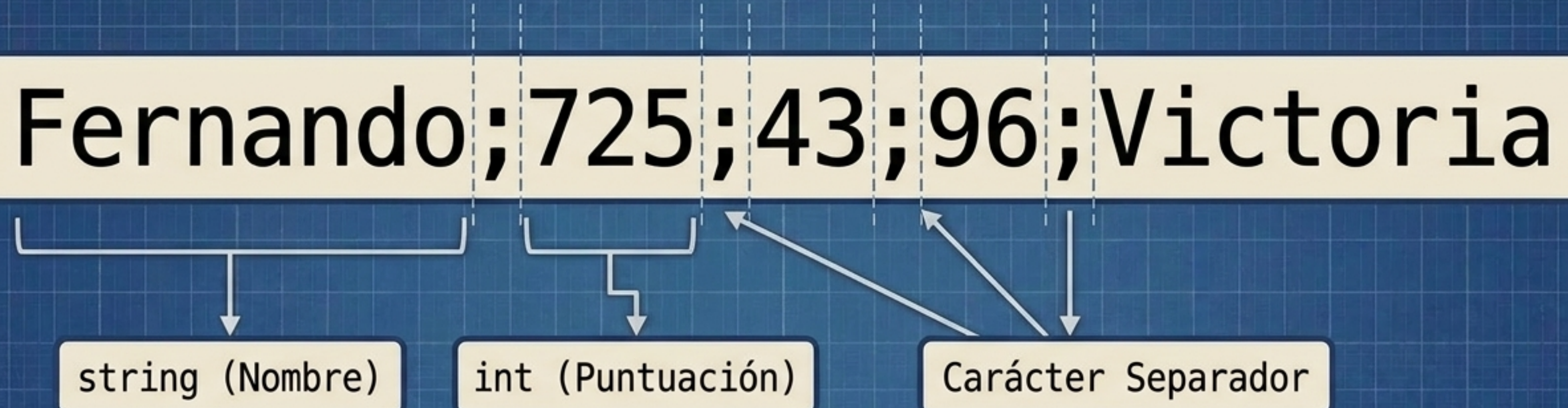
```
if (!Directory.Exists(CARPETA_NIVELES))
```

```
Directory.CreateDirectory()  
→ Generar desde código interno
```

Un programa robusto no crashea si el entorno falla. Si el usuario borra los archivos de texto, el método `CrearFicherosDeNivelesSiNoExisten` detecta la ausencia y utiliza matrices harcodeadas internamente (`ObtenerTextoNivel1`) para regenerar los archivos físicos al arrancar.

Persistencia de Datos: Guardando el Ranking

Fernando;725;43;96;Victoria



string (Nombre)

int (Puntuación)

Carácter Separador

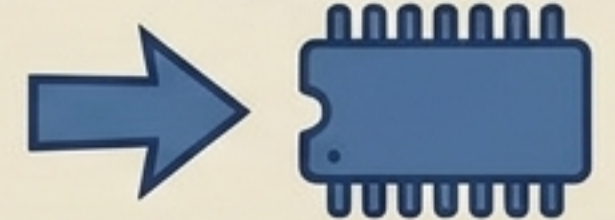
Para guardar el ranking en disco, concatenamos variables de distintos tipos en una sola cadena de texto delimitada por punto y coma. Usamos `File.AppendAllText()` para añadir la nueva partida al final de `ranking.txt` sin destruir el historial anterior.

Mostrando el Ranking: Algoritmos Lógicos



1. Lectura Total

Se lee todo el documento ranking.txt usando `File.ReadAllLines()`.



2. Extracción de Datos

Se aplica `.Split(';')` a cada línea para aislar el índice 1, que contiene el valor numérico de la puntuación.

Name; Score; ...
 ↓
 [1]

3. Comparación Matemática

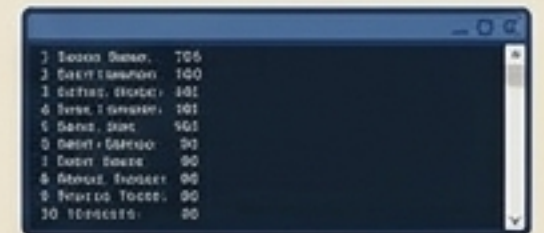
Se utilizan bucles for anidados para comparar y reordenar las puntuaciones (ordenación manual pura, sin usar métodos preconstruidos).

```
for i ...: i++  
  for j > <= > j++  
    ... j++
```



4. Renderizado Final

Se imprimen en la consola únicamente los 10 registros con valores mayores.



Escribir un algoritmo de ordenación manual fortalece la comprensión de índices, matrices y conversiones de tipo (`int.Parse`).



El motor está listo. Ahora el desarrollador eres tú.

→ **Rediseña:** Abre nivel1.txt y crea laberintos endiablados modificando solo texto.

→ **Altera las Reglas:** Cambia las constantes. ¿Qué pasa si la bomba tiene un radio de 3 casillas?

→ **El Gran Reto:** Crea un método para añadir enemigos (E) que persigan al jugador en el bucle.

Programar en C# es construir la arquitectura de la realidad virtual.
Tú escribes las reglas; tú decides cómo jugar.